Semana 2 — File descriptors, sockets e I/O bloqueante

Proyecto: MiniServ — servidor TCP/HTTP multiproceso en C sobre Linux **Pregunta central de la semana:**

> MiniServ V0 imprime y termina. ¿Cómo puede un **mismo proceso** quedarse vivo esperando clientes, y qué ocurre en el kernel cuando `read()` no tiene datos todavía?

Motivación

Problema concreto: MiniServ V0 arranca, escribe `MiniServ iniciado` con `write(1, …)` y termina. Eso demostró procesos y syscalls, pero un servidor real debe **permanecer en ejecución** y **esperar** eventos externos: que alguien se conecte por red y envíe datos.

Por qué importa en Sistemas Operativos: la pregunta ya no es solo "¿quién escribe?", sino "¿qué hace el proceso — y el scheduler — cuando **no hay nada que leer todavía**?". Eso nos lleva a file descriptors como abstracción unificada, I/O bloqueante y sockets como recurso administrado por el kernel, no como "magia de redes".

Manifestación en MiniServ: si convertimos MiniServ en listener TCP, la primera llamada a `accept()` puede **bloquearse** minutos enteros hasta que llegue un cliente. Mientras tanto, el proceso no consume CPU en un bucle vacío: el kernel lo pone en estado **blocked** hasta que haya una conexión pendiente.

Mecanismo del SO que estudiaremos: file descriptors + syscalls de I/O (`read`, `write`, `close`) + creación de sockets (`socket`, `bind`, `listen`, `accept`) + semántica bloqueante de esas llamadas.

Bloque 2 — File descriptors

Qué explicar:

- Un **file descriptor** (fd) es un entero `≥ 0`: índice en la **tabla de descriptores del proceso**.
- El kernel mantiene, por cada fd, una referencia a un **objeto abierto** (archivo, pipe, socket, dispositivo).
- El proceso **no** ve rutas ni direcciones IP en la syscall: solo el número.
- Al nacer, todo proceso hereda **0 = stdin**, **1 = stdout**, **2 = stderr**.

```
Proceso MiniServ
┌──────────────────────────────┐
│ fd 0 ──▶ terminal (stdin)    │
│ fd 1 ──▶ terminal (stdout)   │
│ fd 2 ──▶ terminal (stderr)   │
│ fd 3 ──▶ socket listening    │  ← después de socket+bind+listen
│ fd 4 ──▶ socket conectado    │  ← después de accept()
└──────────────────────────────┘
          │
          │ syscalls read/write/close(fd, ...)
```

```
        ▼
      Kernel
```

Demostración rápida: ejecutar `demo_fd.c` de la Semana 1 o la variante `demo_fds_std.c` (ver sección 11). Mostrar que `write(1, …)` y `write(archivo_fd, …)` usan la **misma** syscall con distinto fd.

> ¿El fd es el archivo en disco?

Respuesta esperada: No. Es un handle en la tabla del **proceso**. Dos procesos pueden tener fd `3` apuntando a recursos distintos. Tras `close(3)`, el kernel puede reasignar el número 3 a otro recurso.

Error frecuente: confundir fd con pathname. Corregir: `open("/tmp/x")` devuelve un entero; las operaciones posteriores usan ese entero.

Herramienta Linux (breve): en otra terminal, con un proceso en ejecución:

```
ls -l /proc/<pid>/fd
```

Mostrar `0 -> /dev/pts/...`, `1 -> ...`. Relacionar con la tabla del proceso.

Bloque 3 — `read`, `write`, `close`

Syscall	Rol principal
`write(fd, buf, count)`	Copia bytes**desde user space hacia** el objeto del fd
`read(fd, buf, count)`	Copia bytes**desde** el objeto del fd **hacia** user space
`close(fd)`	Libera la entrada en la tabla del proceso; decrementa referencia en el kernel

Valores de retorno importantes:

- `write` / `read` devuelven el **número de bytes transferidos**, o `-1` en error (`errno`).
- `read` que devuelve `0` en un socket o pipe: **fin de stream** (peer cerró escritura).
- Siempre verificar retorno; no asumir que se transfirió `count` completo en una sola llamada

Contraste `printf` vs `write`: `printf` bufferiza en user space (stdio); `write` va al kernel inmediatamente. MiniServ usará `write` (o wrappers mínimos) para control explícito.

Demostración: compilar y ejecutar `demo_read_write.c`. Trazar con:

```
strace -e trace=read,write,close ./demo_read_write
```

Preguntas:

1. ¿Cuántas veces cruza la frontera user/kernel un `write` de 100 bytes? (Una syscall; el kernel puede escribir menos de 100 en casos especiales.)
2. ¿Qué pasa si llamamos `read(999, …)` con un fd que no está abierto? (`EBADF`, retorno -1.)

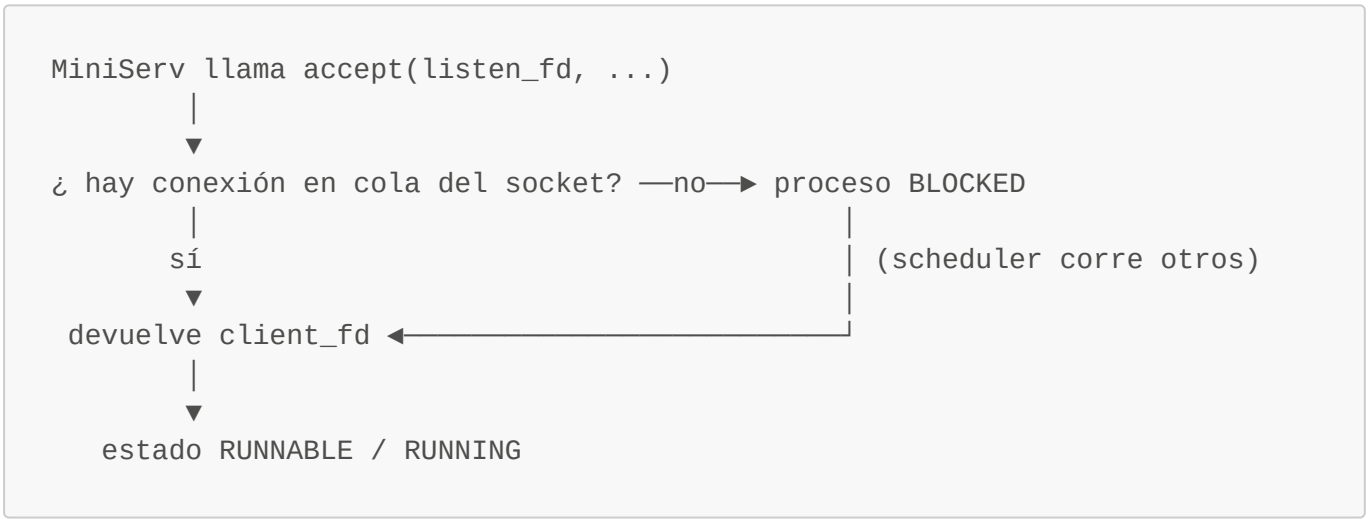
Conexión MiniServ: atender un cliente será un ciclo `read(client_fd, …)` → procesar → `write(client_fd, …)` → eventualmente `close(client_fd)`.

Bloque 4 — I/O bloqueante

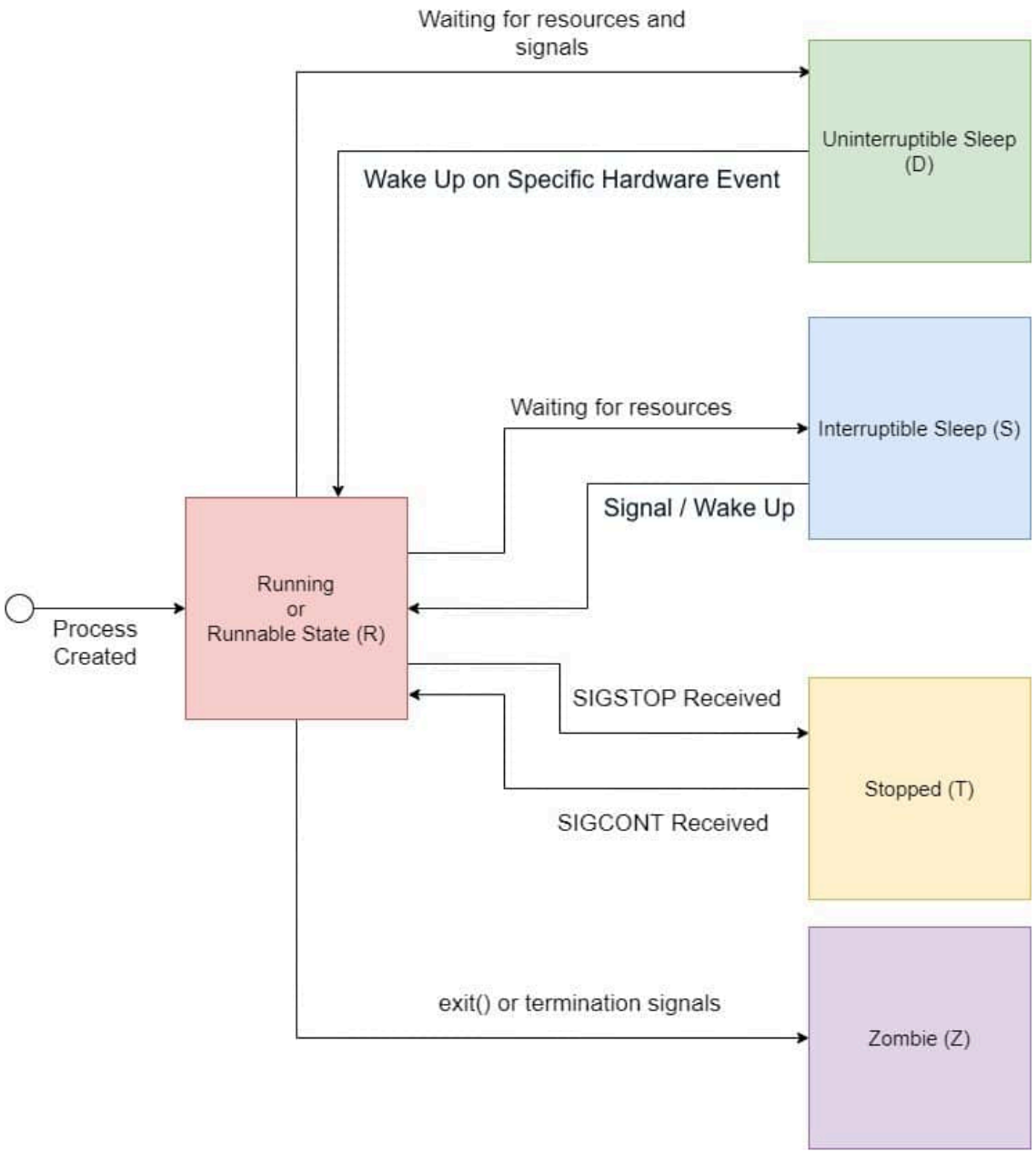
Qué explicar:

- Por defecto, `read` y `accept` en sockets son **bloqueantes**.
- Si no hay datos (o no hay conexión pendiente), la syscall **no retorna** de inmediato: el proceso pasa a estado **blocked**.
- El scheduler ejecuta otros procesos; cuando el kernel recibe datos o una conexión, despierta al proceso y la syscall completa.

Diagrama — estados del proceso durante `accept()`:



Experimento en vivo: `demo_blocking_read.c` — proceso que hace `read(0, …)` y no imprime hasta que el usuario escriba en stdin. En otra terminal: `ps -o pid,state,cmd -p <pid>` → estado `S` (sleeping/interruptible).



Contraste pedagógico:

```
while (1) { }            /* RUNNING: quema CPU */
read(fd, buf, n);       /* BLOCKED si no hay datos: eficiente */
```

Pregunta:

> MiniServ V1 está bloqueado en `accept()`. Llega un cliente. ¿MiniServ consume CPU mientras esperaba?

Respuesta: No de forma significativa en estado blocked. El costo aparece al despertar y procesar la conexión.

Error frecuente: "bloqueante = el programa se congeló/crasheó". Corregir: es comportamiento **normal** y deseado en un servidor secuencial.

Conexión profesional (breve): servidores de producción combinan blocking, non-blocking y multiplexación (`poll`/`epoll`, lo véremos después). Empezar con blocking hace visible el modelo de estados del proceso.

Bloque 5 — Sockets

> Un **socket** es un objeto del kernel para comunicar bytes entre endpoints; en el proceso aparece como **file descriptor** y se usa con `read`/`write`/`close` como un archivo o stdin.

No es un curso de redes; nos interesa **cómo el kernel representa una conexión** y qué syscalls crean y usan esos fds.

Secuencia servidor TCP (IPv4, simplificada):

```
socket()   → crea fd de socket (aún sin puerto local)
bind()     → asocia dirección/puerto local al fd
listen()   → marca fd como "escucha conexiones entrantes"
accept()   → bloquea hasta conexión; devuelve NUEVO fd conectado
read/write → sobre el fd devuelto por accept (no sobre listen_fd)
close()    → liberar fds cuando ya no se usen
```

Dos fds importantes:

fd	Rol
`listen_fd`	Puerta de entrada: solo`accept`
`client_fd`	Conversación con**un** cliente concreto

Mínimo de redes:

```
127.0.0.1:8080  ← dirección IP + puerto (endpoint local)
      +
AF_INET, SOCK_STREAM  ← familia IPv4, byte stream confiable (TCP)
```

Demostración (en dos pasos): `demo_tcp_server.c` (echo con read/write). Levantar servidor, conectar con:

```
nc 127.0.0.1 8080
# o
echo "hola" | nc 127.0.0.1 8080
```

Trazar:

```
strace -e trace=network,socket,bind,listen,accept,read,write,close
```

```
./demo_tcp_server
```

Herramientas en paralelo:

```
ss -ltnp | grep 8080      # socket en LISTEN y proceso dueño
lsof -i :8080             # qué proceso tiene el puerto
```

Preguntas:

1. ¿Por qué `accept` devuelve un **nuevo** fd en lugar de reutilizar `listen_fd`? (Separar escucha de la conversación con un cliente; el listen_fd sigue disponible para más conexiones.)
2. ¿Qué syscall bloquea si no hay clientes? (`accept`, no `listen`.)

Error frecuente: hacer `read(listen_fd, …)` para obtener datos del cliente. Incorrecto: hay que `accept` primero.

Historia (breve): BSD Unix unificó sockets con el modelo "todo es un archivo" (~1983). Linux implementa la misma API POSIX que usará MiniServ.

Bloque 6 — Laboratorio MiniServ V1

Estado inicial: MiniServ V0 — imprime y termina.

Problema: no escucha puerto ni atiende clientes.

Mecanismo: socket + bind + listen + accept + read/write + close.

Cambio arquitectónico:

```
MiniServ V0 (main → printf → exit)
        │
        ▼
   necesita esperar clientes sin quemar CPU
        │
        ▼
   blocking accept + read/write sobre fds
        │
        ▼
MiniServ V1 (loop: accept → echo → close client_fd)
```

Código de referencia: `miniserv/v1/src/server.c` (milestone entregable en carpeta `v1/`). Extensión HTTP secuencial: `miniserv/v2/` (misma arquitectura, respuesta mínima a `GET`).

Esqueleto guía para implementar en clase (completar manejo de errores):

```c
/* miniserv/v1/src/server.c — MiniServ V1, servidor TCP secuencial echo */
#define _POSIX_C_SOURCE 200809L

#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define PORT 8080
#define BACKLOG 8
#define BUF_SIZE 4096

static void die(const char *msg)
{
    perror(msg);
    _exit(1);
}

int main(void)
{
    int listen_fd, client_fd;
    struct sockaddr_in addr;
    socklen_t addrlen = sizeof(addr);
    char buf[BUF_SIZE];
    ssize_t n;

    listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0)
        die("socket");

    int opt = 1;
    if (setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))
< 0)
        die("setsockopt");

    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);

    if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
        die("bind");
    if (listen(listen_fd, BACKLOG) < 0)
        die("listen");

    const char *banner = "MiniServ V1 escuchando en puerto 8080\n";
    if (write(STDERR_FILENO, banner, strlen(banner)) < 0)
        die("write");
```

```
    for (;;) {
        client_fd = accept(listen_fd, (struct sockaddr *)&addr, &addrlen);
        if (client_fd < 0) {
            if (errno == EINTR)
                continue;
            die("accept");
        }

        while ((n = read(client_fd, buf, sizeof(buf))) > 0) {
            ssize_t off = 0;
            while (off < n) {
                ssize_t w = write(client_fd, buf + off, (size_t)(n - off));
                if (w < 0)
                    die("write");
                off += w;
            }
        }
        if (n < 0)
            perror("read");

        close(client_fd);
    }
    /* unreachable */
}
```

Actividad guiada:

1. Actualizar `Makefile` si hace falta (mismos flags; puede añadirse `-D_POSIX_C_SOURCE`).
2. Compilar y ejecutar `./miniserv`.
3. Terminal 2: `nc 127.0.0.1 8080`, escribir texto, ver echo.
4. Terminal 3: conectar **segundo** cliente mientras el primero mantiene la conexión — observar que **espera** (limitación secuencial).
5. `strace -p <pid>` mientras llega un cliente: ver secuencia `accept` → `read` → `write`.

Observación pedagógica clave: con dos clientes, el segundo queda en cola en el kernel hasta que el primero cierra. **No es bug:** es la semántica de un servidor secuencial con un solo hilo de ejecución. Semana 3: `fork()`.

Bloque 7 — Ejercicios, predicción y cierre de clase

Actividad de predicción

Código:

```c
int fd = open("/tmp/x", O_RDONLY);
char buf[4];
ssize_t a = read(fd, buf, 4);
ssize_t b = read(fd, buf, 4);
printf("a=%zd b=%zd\n", a, b);
```

Archivo `/tmp/x` contiene exactamente 6 bytes.

Preguntas: ¿Valores de `a` y `b`? ¿Posición en el archivo tras cada `read`?

Respuesta: `a=4`, `b=2`. El kernel mantiene un **offset** por fd abierto; cada `read` avanza. En sockets TCP el offset no es visible como en archivos, pero la idea de "stream de bytes" es análoga.

Determinismo: tamaños sí; timing de red no.

---

3 preguntas conceptuales (cierre)

1. ¿Por qué `close(client_fd)` no cierra el servidor completo?

Respuesta: Solo libera el fd de **esa** conexión. `listen_fd` sigue abierto y el loop puede llamar `accept` de nuevo.

2. ¿Un servidor bloqueado en `accept()` está "haciendo nada útil"?

Respuesta: Está esperando trabajo I/O-bound de forma eficiente. Útil para un solo cliente; insuficiente para concurrencia (Semana 3).

3. ¿`socket()` ya abre una conexión de red?

Respuesta: No. Crea un endpoint del kernel. Conexión implica `connect` (cliente) o `accept` (servidor tras `listen`).

Qué problema resuelve desde Sistemas Operativos

Sin sockets (o mecanismos equivalentes), cada programa tendría que hablar directamente con el hardware de red o con drivers propios. El kernel concentra:

- direcciones y puertos;
- colas de bytes (buffers de envío/recepción);
- el protocolo de transporte (en MiniServ usamos **TCP**, stream confiable de bytes);
- la decisión de **cuándo** un `read` o `accept` puede desbloquearse.

Desde user space solo vemos: **entero fd** + **syscalls**. El resto es responsabilidad del kernel — igual que con `write(1, …)` y la terminal.

Cómo se ve en el proceso vs. en el kernel

```
Proceso MiniServ (user space)
┌──────────────────────────────────────────┐
```

```
  │  fd 3  ──►  socket en modo LISTEN   │  ← escucha en puerto 8080
  │  fd 4  ──►  socket conectado (TCP)  │  ← un cliente concreto
  └─────────────────────────────────────┘

               │ read / write / accept
               ▼
Kernel
  ┌─────────────────────────────────────┐
  │  struct socket, buffers, cola accept │
  │  stack TCP (estado de conexiones)    │
  │  NIC / loopback si hay tráfico real  │
  └─────────────────────────────────────┘
```

El proceso **no** ve direcciones IP ni puertos como "variables mágicas" en cada `read`: ve **fds**. La dirección y el puerto se configuran una vez con `bind` (servidor) o `connect` (cliente); la conversación diaria es `read`/`write` sobre el fd conectado.

Socket no es lo mismo que puerto ni que conexión

Concepto	Qué es
Socket	Objeto del kernel; en el proceso, un fd
Puerto (p. ej. 8080)	Número lógico en la dirección local del socket; recurso compartido del sistema
Conexión TCP	Par de endpoints acoplados; en el servidor suele materializarse como **nuevo fd** tras `accept()`
`listen_fd`	Socket pasivo: solo acepta conexiones entrantes
`client_fd`	Socket conectado: por aquí van los bytes de **un** cliente

Error frecuente: decir "el puerto 8080 es un fd". El puerto se **asocia** al socket con `bind`; el fd es el handle del proceso hacia ese socket.

Relación con file descriptors e I/O bloqueante

- Crear socket: `socket()` → nueva fila en la tabla de fds del proceso.
- Esperar cliente: `accept(listen_fd, …)` puede **bloquear** (proceso en estado sleeping) hasta que el kernel tenga una conexión lista.
- Intercambiar datos: `read`/`write` sobre el fd conectado; también pueden bloquear si no hay bytes disponibles o el buffer de envío está lleno.

Por eso MiniServ V1 — aunque "sea un servidor de red" — es sobre todo un estudio de **procesos**, **fds** e **I/O bloqueante**: el mismo patrón que `read(0, …)` en stdin, con otro tipo de objeto detrás del fd.

Qué necesitamos dominar esta semana sobre redes

Basta con tres ideas mínimas:

1. **127.0.0.1** — loopback (tráfico que no sale de la máquina; útil en laboratorio).
2. **Puerto** — etiqueta para multiplexar muchos servicios en una IP.

3. **TCP stream** — secuencia de bytes sin mensajes delimitados por el kernel (un `read` puede traer parte de lo enviado).

El foco sigue siendo: **¿qué syscalls ejecuta el proceso y qué hace el kernel cuando bloquea o despierta?**

Historia mínima (contexto)

La API de **sockets BSD** (~1983) unificó la programación de red con el modelo Unix. Linux expone la misma familia de syscalls (`socket`, `bind`, `listen`, `accept`, …) que usará MiniServ durante el semestre.

9.6 Syscalls de servidor sobre un socket

Secuencia típica (servidor TCP):

- `socket`: crea endpoint en el kernel; devuelve fd (aún sin puerto publicado).
- `bind`: asocia dirección IP + puerto local al fd (recurso del sistema).
- `listen`: socket pasivo; el kernel mantiene cola de conexiones pendientes.
- `accept`: bloquea hasta una conexión lista; devuelve **nuevo fd** para ese cliente.
- `read` / `write`: bytes sobre el fd conectado (no sobre `listen_fd`).
- `close`: libera fds; en V1 se cierra `client_fd` tras cada cliente y `listen_fd` sigue abierto en el loop.

MiniServ V1: un solo proceso ejecuta toda la secuencia; un cliente activo bloquea el siguiente `accept` hasta `close(client_fd)`.

Errores frecuentes: olvidar `htons`/`htonl` al llenar `struct sockaddr_in`; hacer `read(listen_fd, …)` en lugar de `accept` + `read(client_fd, …)`; omitir `close` y agotar fds (Semana 14).

Demostraciones en C

Los archivos están en `programas_ejemplo/`. Resumen:

11.1 `demo_fds_std.c` — fds estándar

Objetivo

Mostrar stdin/stdout/stderr como fds 0, 1, 2 y escritura con `write`.

Código completo

Ver archivo `programas_ejemplo/demo_fds_std.c`.

Compilación

```
gcc -Wall -Wextra demo_fds_std.c -o demo_fds_std
```

Ejecución

```
./demo_fds_std
```

Salida esperada

```
stdin=0 stdout=1 stderr=2
esto va a stdout (fd 1)
esto va a stderr (fd 2)
```

No determinismo

Ninguno en PIDs ni orden de líneas.

Explicación paso a paso

1. Imprime constantes `STD*_FILENO`.
2. Cada `write` usa distinto fd → misma syscall, distinto destino en kernel.
3. Tres traps a kernel (`write`).
4. Terminal driver o pipe según redirección.

11.2 `demo_read_write.c` — copia archivo con read/write

Objetivo

`open` → `read` loop → `write` loop → `close`; mismas syscalls servirán para sockets.

Compilación

```
gcc -Wall -Wextra demo_read_write.c -o demo_read_write
```

Ejecución

```
./demo_read_write /etc/hostname /tmp/copia-hostname
cat /tmp/copia-hostname
```

Salida esperada

Contenido idéntico al archivo origen (p. ej. `ubuntu`).

No determinismo

Ninguno.

Explicación paso a paso

1. `open` devuelve fd de lectura.
2. `read` copia desde page cache del kernel al buffer user.
3. `write` copia al archivo destino.
4. `read` retorna 0 al EOF → termina loop.
5. `close` libera entradas.

11.4 `demo_polling_read.c` — Polling para obtner información

Objetivo

Observar proceso en estado de ejecución mientras `read(0, …)` espera. Alto consumo de procesador.

Compilación

```
gcc -Wall -Wextra demo_polling_read.c -o demo_polling_read
```

Ejecución

```
./demo_polling_read
# escribir una línea y Enter
```

Salida esperada

```
Esperando entrada (read bloqueante en stdin)...
Leí 5 bytes: hola
```

No determinismo

Timing depende del usuario.

Explicación paso a paso

1. Mensaje inicial con `write(2, …)`.
2. modificamos la respuesta de`read(0, …)` para que consulte constantemente por los datos, no bloquea → proceso.
3. Observar el estado en ejecución y el alto consumo de recursos
4. Usuario escribe → kernel manda datos al proceso → `read` retorna con datos.
5. Segundo `write` muestra resultado.

11.3 `demo_blocking_read.c` — blocked en stdin

Objetivo

Observar proceso en estado sleeping mientras `read(0, …)` espera.

Compilación

```
gcc -Wall -Wextra demo_blocking_read.c -o demo_blocking_read
```

Ejecución

```
./demo_blocking_read
# escribir una línea y Enter
```

Salida esperada

```
Esperando entrada (read bloqueante en stdin)...
Leí 5 bytes: hola
```

No determinismo

Timing depende del usuario.

Explicación paso a paso

1. Mensaje inicial con `write(2, …)`.
2. `read(0, …)` bloquea → proceso `S` en `ps`.
3. Usuario escribe → kernel despierta proceso → `read` retorna.
4. Segundo `write` muestra resultado.

11.5 `demo_tcp_server.c` — TCP 2 (echo)

Objetivo

Secuencia completa socket/bind/listen/accept/read/write/close con echo.

Compilación

```
gcc -Wall -Wextra demo_tcp_server.c -o demo_tcp_server
```

Ejecución

Terminal 1:

```
./demo_tcp_server
```

Terminal 2:

```
echo hola | nc 127.0.0.1 9090
```

Salida esperada (servidor)

```
listen_fd=3
accept → client_fd=4
read 5 bytes, echo OK
```

(PIDs y números de fd pueden variar.)

No determinismo

Números de fd; orden si varios clientes compiten.

Explicación paso a paso

1. `socket` crea fd.
2. `bind`/`listen` registran puerto 9090 en kernel.
3. `accept` bloquea hasta `nc`.
4. `read`/`write` en `client_fd`.
5. `close` ambos fds al terminar demo (MiniServ mantiene `listen_fd`).

---

12. Herramientas Linux

`strace`

Observar: secuencia `socket`, `bind`, `listen`, `accept`, `read`, `write`, `close` en MiniServ.

```
strace -f -e trace=network,socket,bind,listen,accept,read,write,close
./miniserv
```

Identificar alumno: `accept` bloqueante aparece como llamada que tarda; tras conectar con `nc`, retorna con fd nuevo.

`ss`

Observar: socket en estado LISTEN y conexiones ESTAB.

```
ss -ltnp | grep 8080
ss -tnp  | grep miniserv
```

Relación: muestra objetos de red del kernel asociados al proceso.

`lsof`

```
lsof -i :8080
lsof -p <pid>
```

Identificar: PID dueño del puerto; fds abiertos (`TCP *:8080 (LISTEN)`).

`ps`

```
ps -o pid,state,wchan:20,cmd -p <pid>
```

Observar: estado `S` y `wchan` relacionado con `accept` o `read` cuando bloquea.

`/proc/<pid>/fd`

```
ls -l /proc/<pid>/fd
```

Identificar: symlinks a `socket:[inode]` para fds de red.

---

13. Integración con MiniServ

Estado inicial

MiniServ V0: compila, imprime `MiniServ iniciado`, termina. Conocemos procesos, syscalls y `write` a fd 1.

Problema

No escucha puerto, no espera clientes, no demuestra I/O bloqueante ni fds de red.

Mecanismo del Sistema Operativo

File descriptors + I/O bloqueante + API de sockets POSIX.

Cambio arquitectónico

```
MiniServ V0 (proceso efímero)
      |
      ▼
debe quedarse vivo y esperar conexiones sin busy-wait
      |
      ▼
blocking I/O + sockets (listen_fd / client_fd)
      |
      ▼
MiniServ V1 (servidor TCP secuencial echo)
```

Versión del proyecto: milestone **V1** (TCP echo). La carpeta `miniserv/v2/` añade HTTP secuencial como paso intermedio antes de `fork` en V3.

16. Errores y misconceptions

"`read` siempre llena el buffer"

Incorrecto. Puede devolver menos bytes de los pedidos (archivos y sockets). Hay que usar el valor de retorno.

"Cerrar MiniServ con Ctrl+C es lo mismo que `close(listen_fd)`"

Ctrl+C envía señal (Semana 5). `close` solo libera el fd. Hoy aceptamos Ctrl+C como atajo; cierre ordenado vendrá después.

"El puerto es un fd"

El **puerto** es número lógico en la dirección del socket. El **fd** es handle del proceso hacia el objeto socket en el kernel.

"`accept` lee datos del cliente"

`accept` crea conexión y devuelve fd. Los datos se leen con `read` sobre ese fd.

"Blocking I/O = malo"

Es ineficiente **solo** cuando necesitamos atender muchos clientes simultáneos con un solo hilo. Es el modelo correcto para empezar y para cargas simples.

"Sockets no son file descriptors"

En Unix/Linux, los sockets **son** fds: `read`/`write`/`close` aplican (con semántica de stream).

---

17. Conexión profesional

- **Apache/Nginx (modo inicial):** aceptan conexión y atienden; la evolución a worker pools (Semana 11) replica decisiones de servidores reales.
- `ss`/`lsof`**:** operadores usan estas herramientas para "¿quién tiene el puerto 443?" — mismo objeto kernel que estudiamos.
- **PostgreSQL:** cada sesión es conexión TCP → fd en el postmaster/backend; patrón accept/read/write.
- **Docker:** publicar puerto (`-p 8080:8080`) reenvía tráfico al fd del proceso en un namespace de red; el fd sigue siendo recurso del kernel visto por el contenedor.
- **systemd socket activation:** el supervisor puede pasar un `listen_fd` ya abierto al servicio — herencia de fds (anticipación Semana 4).

---

18. Historia y fun facts

1. **"Everything is a file"** (Thompson/Ritchie, Unix): unificar dispositivos, pipes y archivos bajo fds facilitó que MiniServ use `read`/`write` en TCP igual que en `/etc/hostname`.
2. **Sockets BSD (1983):** antes, APIs de red eran más ad hoc; sockets se integraron al modelo Unix y POSIX los estandarizó.
3. `nc` **(netcat):** herramienta de 1996 aún útil para generar tráfico controlado sin escribir un cliente — ideal para laboratorio de SO.
4. **Puerto bien conocidos:** 8080 como alternativa HTTP de desarrollo; en producción el kernel distingue listeners por `(IP, puerto, protocolo)`.

NOTAS sobre HTTP

https://www.freecodecamp.org/news/what-is-http/